

多智能体共享内存系统中的 层次一致性模型与冲突解决机制

멀티 에이전트 공유 메모리 시스템에서의 계층형 일관성 모델 및 충돌 해소 메커니즘 설계

YU BAOQI (위바오치)
2026年 6月

研究背景与研究动机

연구 배경 및 연구 동기

背景与动机 | 배경 및 동기

■ LLM多智能体协作成为主流范式

LLM 기반 멀티 에이전트 협업이 주류 패러다임으로 부상

■ 共享内存是Agent信息同步的核心基础设施

공유 메모리는 에이전트 간 상태 동기화의 핵심 인프라

■ 三大冲突类型威胁共享内存的可靠性:

세 가지 충돌 유형이 공유 메모리의 신뢰성을 위협:

① 可见性冲突 (Visibility) — Agent读取到过期数据

① 가시성 충돌 — 에이전트가 오래된 데이터를 읽음

② 顺序性冲突 (Ordering) — 旧版本覆盖已更新的新版本

② 순서성 충돌 — 과거 버전이 최신 값을 덮어씀

③ 语义冲突 (Semantic) — 互为矛盾的文本结论

③ 의미적 충돌 — 상반된 자연어 결론이 모순됨

■ 现有研究空白: 缺乏形式化定义 + 策略系统性比较 + 语义层面处理

기존 연구 한계: 형식적 정의 부족, 전략 비교 미비, 의미 충돌 처리 미흡

研究问题与研究目标

연구 문제 및 연구 목표

📋五大研究问题 | 5가지 연구 문제 (RQ1~RQ5)

RQ1: 能否精确检测并解决三种冲突类型?

RQ2: 层次一致性模型是否优于单一一致性模型?

RQ3: 时间戳/信任度/领导者/Judge 四种策略中哪种最有效?

RQ4: LLM Judge Agent 能否有效检测语义冲突?

RQ5: 消融实验能否分离各模块的独立贡献度?

🎯研究目标 | 연구 목표

① 形式化定义三种冲突类型

② 设计层次一致性模型

③ 集成三阶段冲突解决

④ 实现8组对比实验

⑤ 消融实验量化模块贡献

📐方法: Design Science

💻原型: Python + SQLite

🗣️LLM: DeepSeek-chat

➡ 总: 720次实验运行

实验环境 (1/3): 系统架构总览

실험 환경 (1/3): 시스템 아키텍처 개요

六层架构 | 6계층 아키텍처

▶ Agent 层

Planner / Executor / Reviewer / JudgeAgent

▶ Conflict 层

ConflictDetector → ConflictResolver (3-Phase)

▶ Consistency 层

Strong | Eventual | Hierarchical

▶ Strategy 层

Timestamp | TrustScore | Leader | Judge

▶ Memory 层

SQLite (WAL): Working Memory + Long-Term Memory

▶ Metrics 层

MetricsCollector → Reporter → CSV

数据流 | 데이터 흐름

- ① 场景生成 → 冲突场景
- ② Agent写入 → 共享内存
- ③ 冲突检测 → 版本比对
- ④ 结构过滤 → 规则筛选
- ⑤ Judge → LLM语义判断
- ⑥ 最终决策 → 写回内存
- ⑦ 指标收集 → 统计分析

技术栈 | 기술 스택

Python 3.12 · NumPy
SQLite (WAL mode)
DeepSeek-chat API
OpenAI SDK (兼容)
Windows 11 环境
GitHub 开源仓库

实验规模

8 个比较组
3 种冲突场景
30 次迭代/场景
—
= 720 次运行
—
3 个 Agent:
Planner
Executor
Reviewer
—
随机种子: 42

实验环境 (2/3): 代码结构与核心模块

실험 환경 (2/3): 코드 구조 및 핵심 모듈

核心类与方法 | 핵심 클래스 및 함수

📁 prototype/ 项目结构

```
prototype/
├── config.py      # 配置常量与API密钥
├── main.py        # 入口：运行全部实验
├── agents/
│   ├── base.py   # BaseAgent (读写/信任度追踪)
│   ├── roles.py  # Planner/Executor/Reviewer
│   └── judge.py   # JudgeAgent (LLM语义判定)
├── memory/
│   ├── models.py # MemoryEntry, ConflictRecord
│   └── store.py  # SQLite共享内存存储 (WAL)
├── conflict/
│   ├── detector.py # 结构性冲突检测器
│   └── resolver.py # 3阶段冲突解决管道
├── consistency/
│   ├── strong.py  # 强一致性 (锁机制)
│   ├── eventual.py # 最终一致性 (非阻塞)
│   └── hierarchical.py # 层次一致性 (路由)
├── strategies/
│   ├── timestamp.py # 时间戳优先策略
│   ├── trust_score.py # 信任度优先策略
│   └── leader.py    # 领导者仲裁策略
├── experiments/
│   ├── runner.py   # ExperimentRunner 主循环
│   ├── scenarios.py # 3种场景 + 15组矛盾语句对
│   ├── groups.py   # 8组比较组 ResolverConfig
│   ├── metrics.py  # MetricsCollector 指标聚合
│   └── reporter.py  # CSV导出与结果打印
```

► SharedMemoryStore

SQLite CRUD, 版本管理, 冲突日志 [store.py]

► ConflictDetector

版本对比检测 visibility/ordering [detector.py]

► ConflictResolver.resolve()

4阶段: 检测→过滤→Judge→决策 [resolver.py]

► ResolverConfig

布尔标志控制8组配置开关 [resolver.py:L19]

► JudgeAgent.judge()

DeepSeek API → JSON解析 [judge.py]

► ScenarioGenerator

3种场景 × 15组矛盾语句 [scenarios.py]

► HierarchicalConsistency

按 memory_type 自动路由 [hierarchical.py]

► ExperimentRunner.run_all()

主循环: 8组 × 3场景 × 30轮 [runner.py]

► build_groups()

返回 8 个 ResolverConfig [groups.py]

实验环境 (3/3): 8组比较组与评价指标

실험 환경 (3/3): 8개 비교군 및 평가 지표

G1 · Baseline

无检测 无策略 无Judge | Last-Write-Wins만 적용

G2 · StrongOnly

仅强一致性, 无策略 | 강한 일관성만, 전략 없음

G3 · Timestamp

时间戳优先 + 预过滤 | 타임스탬프 우선 + PreFilter

G4 · TrustScore

信任度优先 + 预过滤 | 신뢰도 우선 + PreFilter

G5 · Leader

领导者仲裁 + 预过滤 | 리더 중재 + PreFilter

G6 · Ablation1 (NoJudge)

层次一致性+策略, 无Judge | 계층형+전략, Judge 제외

G7 · Ablation2 (NoPreFilter)

层次一致性+策略+Judge, 无预过滤 | 계층형+전략+Judge, PreFilter 제외

G8 · FullProposed ★

全组件: 层次一致性+策略+Judge+预过滤 | 제안 방식: 모든 구성 요소 통합

层次一致性模型设计

계층형 일관성 모델 설계

🔒 工作内存 | Working Memory

键: task_plan, task_status, current_step

一致性: 强一致性 (Strong Consistency)

机制: Lock-based Serialization

→ 所有读写串行执行

→ Agent始终读取最新值

목적: 실시간 협업 정보의 엄격한 정합성 보장

📖 长期内存 | Long-term Memory

键: knowledge_base, decision_log, feedback_history

一致性: 最终一致性 (Eventual Consistency)

机制: Non-blocking Write

→ 冲突时 Last-Write-Wins 收敛

→ 短暂不一致可接受

목적: 축적형 지식의 확장성과 유연성 확보

💡 设计理念与实现 | 설계 철학 및 구현

- ▶ 工作内存访问频繁、错误代价高 → 必须强一致性 | 작업 메모리 = 잠금 기반 직렬화
- ▶ 长期内存积累知识、容忍短暂延迟 → 最终一致性足够 | 장기 메모리 = 비차단 쓰기
- ▶ HierarchicalConsistency 类: 根据 memory_type 自动路由到对应协议
- ▶ upgrade_to_long_term(): 验证后的工作内存条目可安全提升到长期内存

3阶段冲突解决机制

3단계 충돌 해소 메커니즘

Phase 1: 冲突检测 단계1: 충돌 탐지

ConflictDetector 对比 AgentState 版本

- **Visibility:** $\text{seen_ver} < \text{current_ver}$
- **Ordering:** $0 < \text{seen_ver} < \text{current_ver}$

→ 输出: 冲突类型 (visibility/ordering)

Phase 2: 结构预过滤 단계2: 구조적 사전 필터링

三种策略 (择一):

- 时间戳优先: 选最新写入值
- 信任度优先: 选高信任Agent值
- 领导者仲裁: Reviewer规则判断

→ 快速解决结构性冲突

→ 减少不必要的LLM调用

Phase 3: Judge Agent 语义判断 단계3: Judge Agent 의미 판단

DeepSeek-chat API 调用

- JSON格式: {conflict, resolution, reasoning}
- 语义矛盾检测 + 最优值选择

→ 仅对自然语言值调用 (>40字符/含标点)

→ needs_semantic_check() 精确筛选

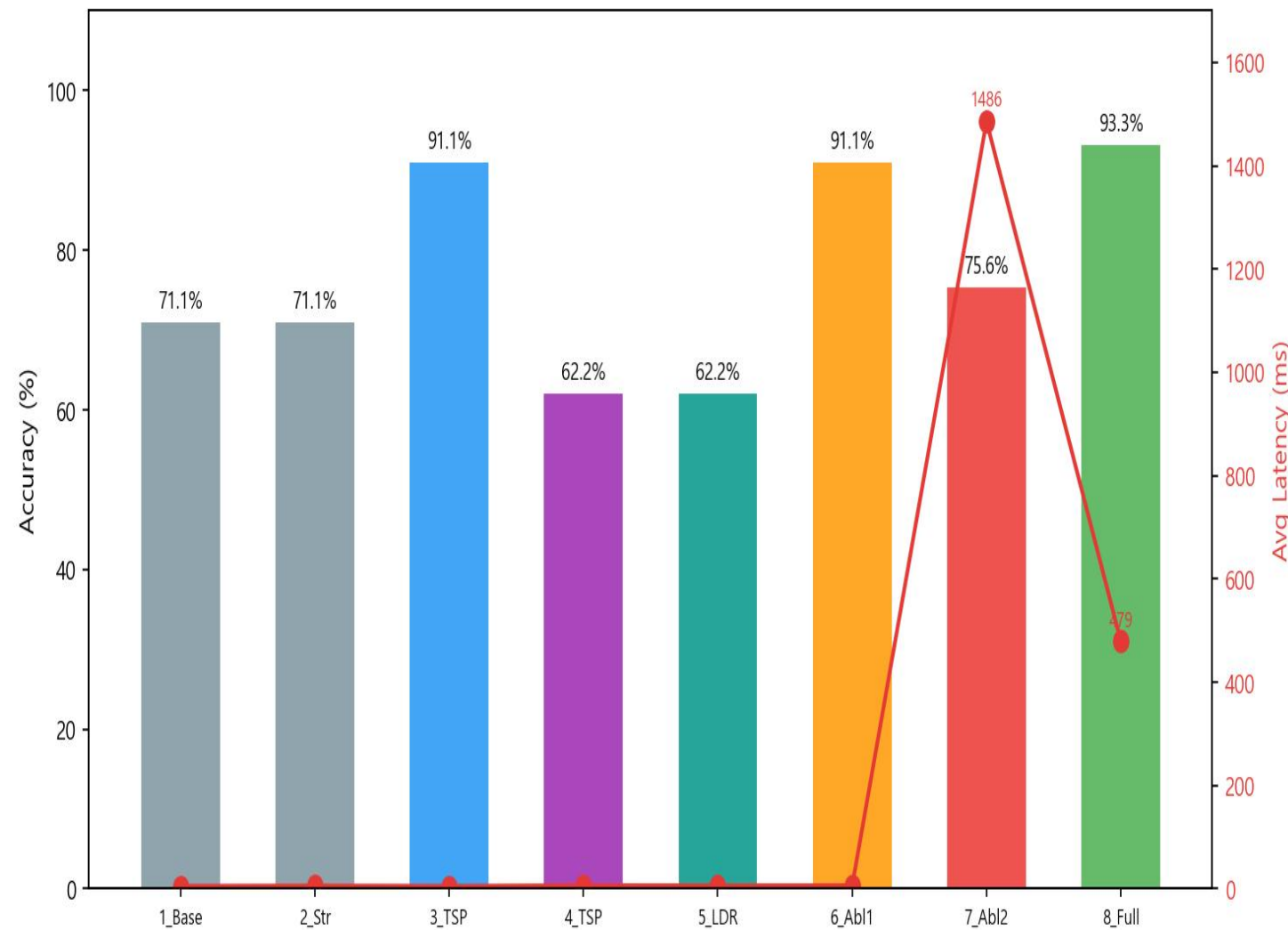
⚡ 决策流程 | 통합 의사결정 (ConflictResolver.resolve)

```
def resolve(key, value_a, value_b):  
    1) detect(agent_state, key) → 结构冲突类型 (visibility / ordering)  
    2) if pre_filter: apply(timestamp | trust_score | leader) → 结构筛选  
    3) if judge and needs_semantic_check(v_a, v_b): judge(key, v_a, v_b) → LLM语义判断  
    4) final = judge_result or structural_result or last_write_wins → 返回最终值
```


实验结果 (1/3): 综合性能对比

실험 결과 (1/3): 종합 성능 비교

Figure 1. Comparison Group Overall Accuracy and Average Latency



🔍 核心发现 | 핵심 발견

☑️ FullProposed 准确率: 93.3%

Baseline (71.1%) 대비 +22.2%p

☑️ 一致性维持率: 100% (G3-G8)

Baseline & StrongOnly: 0%

☑️ 延迟: 478.7ms (FullProposed)

Ablation2 (1485.8ms) 대비 3.1×↓

☑️ 最优结构性策略: Timestamp 91.1%

구조적 충돌에서 가장 효과적

⚠️ TrustScore / Leader: 62.2%

语义冲突仅 26.7% — 单策略局限

💧 冲突检测 = 一致性必要条件

충돌 탐지는 일관성 확보의 필요조건

实验结果 (2/3): 场景分析 & 消融实验

실험 결과 (2/3): 시나리오별 분석 및 소거 실험

Figure 2. Accuracy by Scenario per Comparison Group

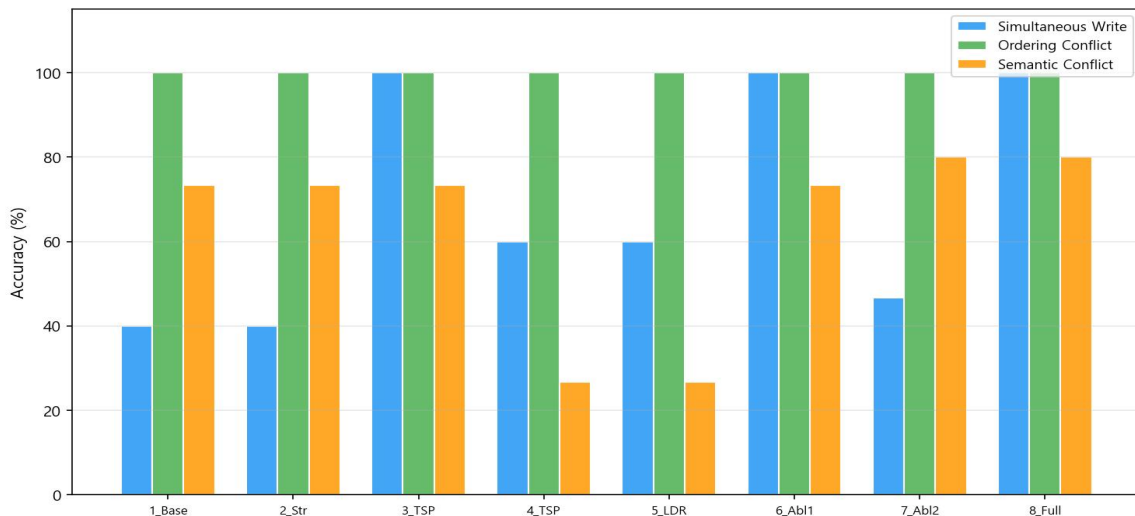


图2: 场景精确度对比 | 그림2: 시나리오별 정확도

Figure 3. Ablation Study: Performance Change vs FullProposed

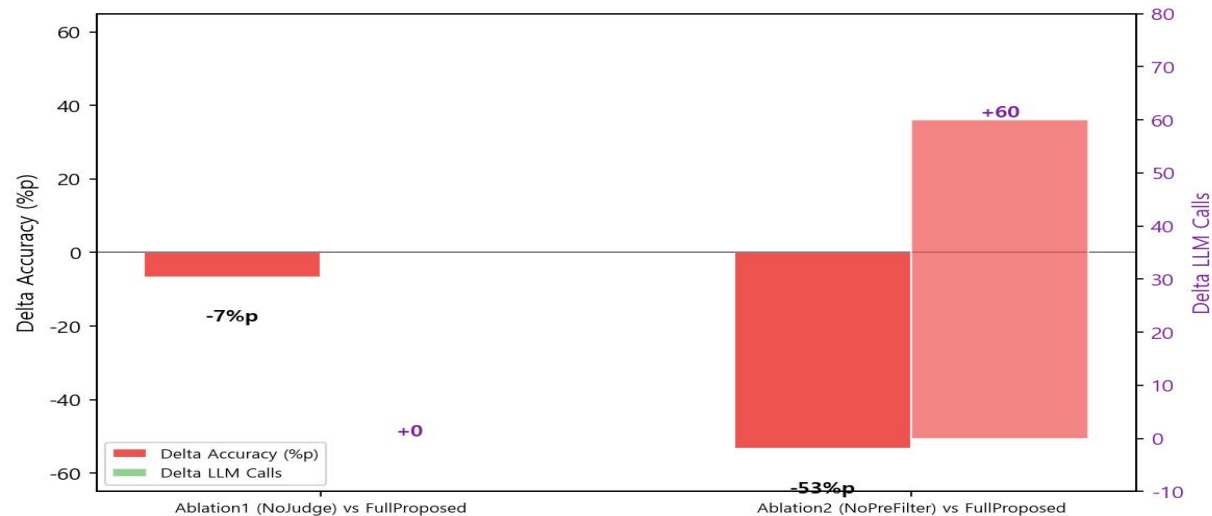


图3: 消融实验结果 | 그림3: 소거 실험 결과

消融实验关键发现 | 소거 실험 주요 발견

■ Ablation1 (NoJudge): 语义冲突准确率 ↓6.67%p

→ Judge Agent 제거 시 의미적 충돌 정확도 6.67%p 하락 → Judge Agent의 의미 충돌 기여도 입증

■ Ablation2 (NoPreFilter): LLM调用 30→90 (+200%), 同时写入准确率仅 46.7%

→ 사전 필터링 제거 시 LLM 호출 3배 증가, 동시 쓰기 정확도 46.7%로 급락

■ 结论: 预过滤与Judge Agent互补, 两者结合达到最优平衡 (准确率93.3% + 延迟478.7ms)

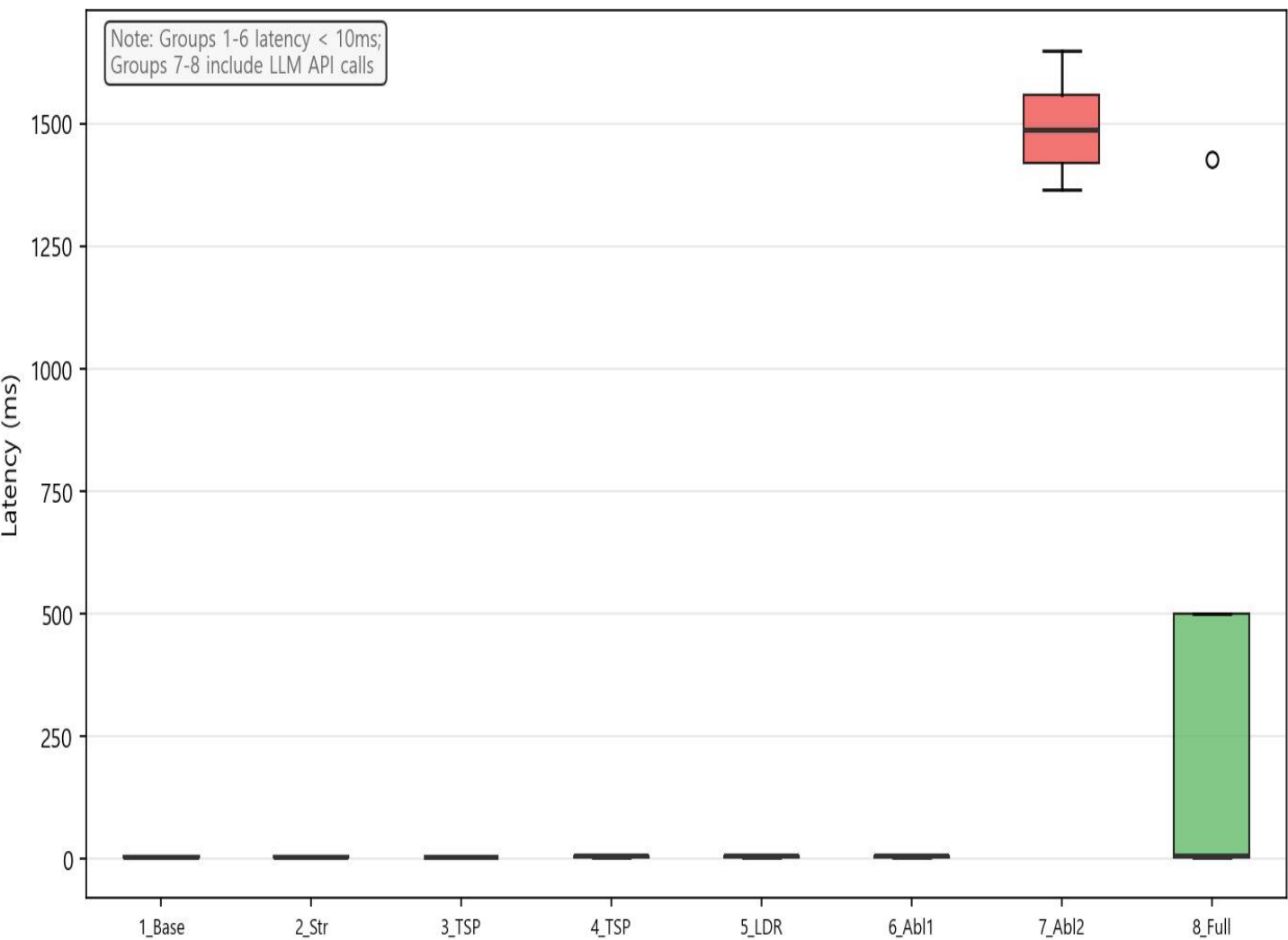
→ 사전 필터링 + Judge Agent 통합이 최적의 정확도-효율성 균형점 달성

实验结果 (3/3): 延迟分析

실험 결과 (3/3): 지연시간 분석

🕒 延迟分析 | 지연시간 분석

Figure 4. Latency Distribution by Comparison Group (Box Plot, N=90)



● 低延迟组 (G1-G6)

全部场景 < 10ms

纯结构性处理, 无LLM调用

● 高延迟组 (G7 Ablation2)

1300-1700ms (全部场景)

每次运行都调用LLM (90次)

● 适中组 (G8 FullProposed)

结构性冲突: < 10ms

语义冲突: LLM调用时增加

平均延迟: 478.7ms

比G7 Ablation2低 3.1倍

💡 预过滤器的实用价值

阻断不必要的LLM调用

降低延迟 · 减少变异性 · 节省成本

🌀五项核心贡献 | 5대 핵심 기여

1. 形式化定义了三种冲突类型及检测条件

가시성·순서성·의미적 충돌의 형식적 정의 및 탐지 조건 구체화

2. 提出并实现了层次一致性模型

작업 메모리(강한 일관성) + 장기 메모리(최종 일관성)의 계층형 설계

3. 集成了三阶段冲突解决管道

충돌 탐지 → 구조적 사전 필터링 → LLM Judge Agent 의 통합 파이프라인

4. 8组对比实验 + 消融实验, 系统验证

8개 비교군과 소거 실험을 통한 모듈별 기여도 정량적 검증

5. 完整方案: 准确率93.3%, 一致性100%, 延迟改善3.1倍

FullProposed: 정확도 93.3% (+22.2%p), 일관성 100%, 지연 3.1배 개선

계층형 일관성 모델과

구조적-의미적 통합

충돌 해소 메커니즘을 통해

멀티 에이전트 공유 메모리의

신뢰성과 효율성을

동시에 향상시켰다.

—

通过层次一致性模型与

结构-语义统一冲突解决机制

同时提升了多智能体

共享内存系统的